

LAB 3: Problem Statement: Public bus live tracking & crowding estimator

NAME: RACHITH M

My Github link: <https://github.com/rachithm88>

SRN: PES2UG24CS392

SECTION: G

Problem Number:24

System: Public Bus Live Tracking & Crowding Estimator

The **Public Bus Live Tracking & Crowding Estimator** is a municipal transit intelligence platform that provides passengers with real-time information about public buses. The system:

- Receives GPS coordinates from active buses.
- Tracks the current location of buses.
- Calculates estimated arrival times (ETA) for upcoming stops.
- Receives passenger/ticketing sensor information.
- Estimates the crowding level of each bus.
- Provides live bus location, ETA, and crowding information to commuters.
- Allows fleet controllers to monitor buses and transit operations.

Key Functional Requirements

ID	Requirement
FR-001	The system shall ingest GPS coordinates from buses at regular intervals.
FR-002	The system shall calculate and update ETA for upcoming bus stops.
FR-003	The system shall estimate passenger crowding levels.
FR-004	The system shall display live bus location and ETA to commuters.
FR-005	The system shall allow fleet controllers to monitor active buses.

Key Non-Functional Requirements

Requirement	Importance
Performance	The system should process telemetry from many buses with low latency.
Scalability	The system should support increasing numbers of active buses.
Availability	Live tracking should remain available even if one subsystem fails.
Security	GPS and operational data must be protected from unauthorized access.
Reliability	ETA and crowding information should be updated accurately and consistently.

Architectural Style Analysis:

Architecture	Advantages for Bus System	Disadvantages for Bus System
Layered	Clear separation between presentation, business logic and data; easy to understand and develop	Scaling individual functions such as GPS processing or ETA calculation is difficult
Microservices	Individual services can scale independently; failure can be isolated; suitable for different processing requirements	More complex deployment and communication; network latency and data consistency must be managed
Client-Server	Simple centralized architecture; easy data management and deployment	Central server can become a bottleneck and single point of failure

Selected Architecture-Microservices Architecture

I chose Microservices Architecture for the Public Bus Live Tracking & Crowding Estimator system.

Reason 1 – Independent Scalability

The GPS tracking, ETA calculation, and crowding estimation components may experience different workloads. Microservices allow individual services to be scaled independently. For example, the GPS Tracking Service can be scaled to handle telemetry from a large number of active buses without unnecessarily scaling the entire application.

Reason 2 – Fault Isolation

The system provides several independent services. If the Crowding Estimator Service becomes temporarily unavailable, the GPS Tracking Service and ETA Calculation Service can continue providing live location and arrival information. This improves the overall reliability of the transit system.

Security Advantage

Microservices provide clear service boundaries, allowing authentication and authorization to be applied to individual APIs. Each service can expose only the data and operations required by other components, reducing unnecessary access to internal system resources.

Performance Benefit

Microservices allow high-load services such as GPS Tracking and ETA Calculation to be scaled independently. This helps the system process frequent real-time telemetry updates and provide low-latency ETA information to commuters without requiring every component to be scaled equally.

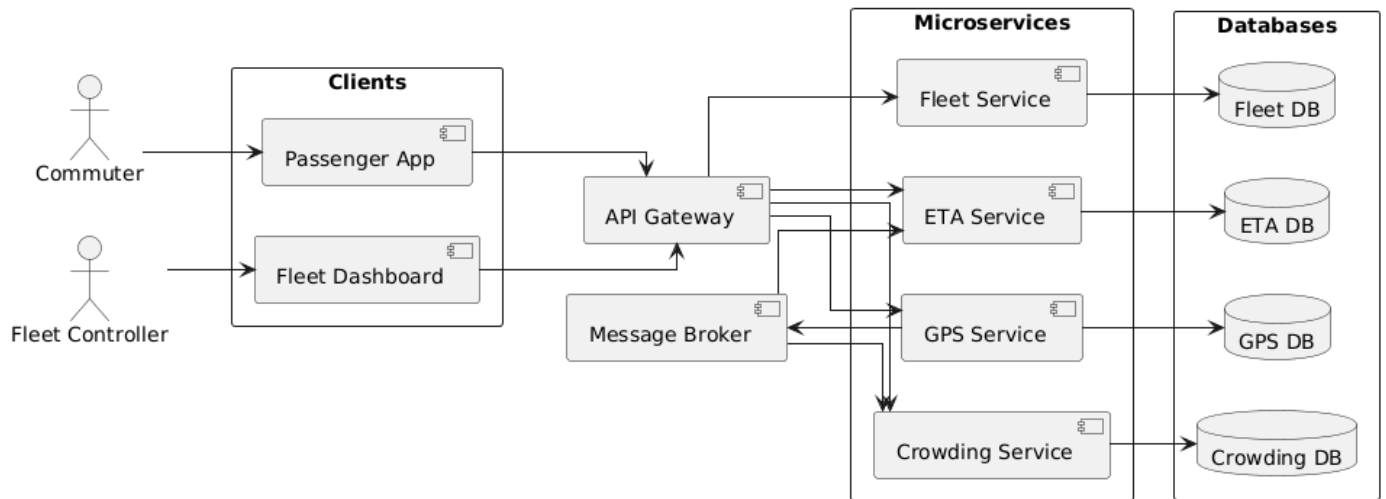
Conclusion

Microservices architecture provides the best balance of scalability, fault isolation, performance, and security for the Public Bus Live Tracking & Crowding Estimator. Although it introduces additional communication and deployment complexity, its ability to independently scale and isolate the major real-time functions makes it suitable for this system.

UML image: (using PlantUML)

Simpler outline image:

Public Bus Live Tracking & Crowding Estimator Microservices Architecture



Detailed image:

Public Bus Live Tracking & Crowding Estimator Microservices Architecture

